Структура кода

Подтемы занятия

- модули в C++
- проблемы монолитного кода
- слоистая архитектура приложений
- организация файлов по папкам
- практические преимущества структурирования

Модули в C++

Модуль – это логически связанная пара файлов:

.hpp файл содержит:

объявления классов и функций, но не их реализацию

.cpp файл содержит:

реализации функций и методов классов

Преимущества

- быстрый анализ назначения класса по .hpp файлу
- для написания зависимых классов можно временно написать только .hpp файл, а реализацию .cpp - позже

Пример модуля

Account.hpp

```
#pragma once
#include <string>

class Account {
    public:
        Account(
            const std::string& owner_name,
            double initial_balance);
        void deposit(double amount);
    private:
        std::string m_owner;
        double m_balance;
}
```

Account.cpp

```
#include "Account.hpp”
#include <iostream>

Account::Account(
    const std::string& owner_name,
    double initial_balance) {
    m_owner = owner_name;
    m_balance = initial_balance;
}

void Account::deposit(double amount) {
    m_balance += amount;
}
```

Проблемы монолитного кода

Монолитный код – это подход к организации программы, при котором вся логика, данные и функции собраны в одном или нескольких крупных файлах без чёткого разделения на интерфейсы (.hpp) и реализации (.cpp).

❌ **Что будет без модульной структуры файлов?**

Дублирование кода

Часто оказывается, что нужная функция уже реализована, но в большом файле её тяжело найти, поэтому проще продублировать

Сложность внесения изменений

Один большой файл на 5000+ строк тяжелее править, чем несколько маленьких

Слоистая архитектура

Цель ***слоистой архитектуры*** – чёткое разделение ответственности кода и обеспечение независимости слоёв.

Интеграционный слой

Содержит логику для считывания или записи данных в базу данных, консоль, файл и т.п.

Бизнес слой

Содержит основные функции для обработки данных. Для обработки данный слой запрашивает данные из интеграционного слоя, но при этом не знает, из какого источника были получены.

Презентационный слой

Содержит логику для отображения данных пользователю. На данном уровне идёт вызов функций бизнес-слоя и отображение их результата

Организация по папкам

Плоская структура файлов в проекте приводит к хаосу при расширении

Для решения проблемы необходимо группировать файлы по их смыслу в папки

Пример

project_root

models — Содержит основные сущности без логики (DTO)

core — Содержит сервисы, функции бизнес-логики

repositories — Содержит функции для запроса или ввода данных

views — Содержит функции для отображения данных

Преимущества структурирования

Легко найти нужный фрагмент кода

Проще сфокусироваться на конкретной задаче, когда нет захламления

Инструменты для создания библиотек лучше работают с папками

Упрощает параллельную работу над одним проектом

Для каждого отдельного модуля проще писать тесты

Конец презентации

Разработчик

Марышев А.В.

Контакты для связи

Телеграм: @XekS_s